

## Propuestas de modificaciones del proyecto 2

Tiempo estimado máximo: 15 minutos

Todas las modificaciones se deben verificar con un archivo de entrada adecuado para cada modificación.

1. Implementar bucle do-while (ejecuta al menos una vez):

Entrada:

```
// Falsa desde el inicio
var x int32 = 10
for {
    fmt.Println(x)
    if !(x < 5) {
        break
    }
}
fmt.Println("Segundo while")
// Comportamiento normal
var i int32 = 0
for {
    fmt.Println(i)
    i++
    if !(i < 3) {
        break
    }
}
```

Salida:

```
10
Segundo
while 0
1
2
```

2. Implementar operador de potencia (\*\*) en las expresiones

Entrada:

```
var result int32 = 2 ** 3
fmt.Println(result)
```

Salida:

```
8
```

3. Se agrega ^^ para XOR lógico (verdadero si exactamente uno es verdadero).

Entrada:

```
var a bool = true
var b bool = false
var c bool = true
```

```

fmt.Println(a  ^^
b)  fmt.Println(a
^^      c)
fmt.Println(b  ^^
c)

```

Salida:

```

true
fals
e
true

```

4. Bucle for con rango inclusivo -> for (i in x..y))

Entrada:

```

for i in 1..4 {
    fmt.Println(i)
}

```

Salida:

```

1
2
3
4

```

5. Un case puede listar varias constantes por medio de rango E1..E2. Coincide si la variable está dentro de algún rango.

Entrada:

```

var score int32 = 85
switch score {
case 90..100:
    fmt.Println("A")
    break
case 80..89:
    fmt.Println("B")
    break
case 70..79:
    fmt.Println("C")
    break
default:
    fmt.Println("F")
}

```

Salida:

```

B

```

6. Operador de comparación por rango: in [E1..E2] y not in [E1..E2] Entrada:
 

```

var x int32 = 10
fmt.Println( x in [5..10] )
fmt.Println( x not in [11..20] )

```

Salida:

tru  
e  
tru  
e

7. Operador de composición de funciones. Permite escribir  $f \circ g(x)$  como  $f(g(x))$ .

```
func doble(x int) int {  
    return x * 2  
}
```

```
func suma1(x int) int {  
    return x + 1  
}
```

```
func main() {  
    r := doble ◦ suma1 (3)  
    fmt.Println(r)  
}
```

Salida:

8

8. Orden de evaluación definido (derecha a izquierda). Define que los argumentos de función se evalúan de derecha a izquierda.

```
func f(x int) int {  
    fmt.Println(x)  
    return x  
}
```

```
func g(a int, b int) int {  
    return a + b  
}
```

```
func main() {  
    r := g(f(1), f(2))  
    fmt.Println(r)  
}
```

Salida:

2

1

3

9. Operador ternario (cond ? E1 : E2). Introduce evaluación condicional dentro de expresiones.

```
func f() int {  
    fmt.Println(1)  
    return 1  
}
```

```
func g() int {  
    fmt.Println(2)
```

```
    return 2
}
```

```
func main() {
    r := true ? f() : g()
    fmt.Println(r)
}
```

Salida:

1  
1

10. Operador de “pipe” (|>). El valor de la izquierda se convierte en el primer argumento de la función de la derecha.

```
func f(x int) int {
    return x + 2
}
```

```
func g(x int) int {
    return x * 3
}
```

```
func h(x int) int {
    return x - 1
}
```

```
func main() {
    r := 2 |> f() |> g() |>
    h()
    fmt.Println(r)
}
```

Salida  
: 11